How LeaseManager works

The complete guide to your project: architecture, data, security, every business flow, the AI, deployment and tests — and the questions a jury is likely to ask

---

LeaseManager — office lease management platform
Final year project · ESPRIT × Quicky Prime · Marwan Hamed

How to read this guide

Parts 1 to 4 are the foundations: what the platform is, how it is built, how its data is shaped and how it keeps organisations apart. Read them first; everything else rests on them.
Parts 5 and 6 walk through every feature as it actually runs, step by step. Part 7 is how the project is started, tested and deployed. Part 8 is a tour of the screens. Part 9 prepares the defense: the questions a jury tends to ask, with answers grounded in the code. Part 10 is a glossary.
Each part ends with an **In one sentence** *box. If you can say those sentences in your own words, you can explain the whole project.*

Accuracy

Every statement was checked against the code on 28 October 2026 — file names are given in `monospace` *so you can open them. Counts that depend on the date (bookings, invoices) come from the demo data and move a little each day.*

Contents

1 · The big picture — what it is, who uses it, the business chain, the demo world

2 · Architecture — the four containers, the life of a request, back end, front end

3 · The data model — 25 tables, the two owners of a record, every status

4 · Security and isolation — sign-in, the five checks, keeping competitors apart, the proof

5 · The business flows — visitor, onboarding, the in-person intake, bookings, leases, billing, maintenance

6 · The AI features — maintenance triage, the space finder, document reading, two help assistants

7 · Running, testing, deploying — Docker, nginx, the demo data, tests, CI, commands

8 · A tour of the screens — what each role sees

9 · Questions for the defense — likely questions, with answers

10 · Glossary — the words used in this guide

The project in numbers

	Figure
Organisation	3 levels of organisation, 9 roles
Database	PostgreSQL 13.5 · 25 tables · 36 migrations
Back end	NestJS 11 · 37 modules · 30 controllers · 221 route handlers, all in Swagger
Front end	React 19 + Vite 7 · 55 page components · 79 routes (redirects included)
Scheduled jobs	3 — reminders at 08:00, onboarding emails at 09:00, marketplace listing sync every 15 minutes
AI	1 configurable client · 5 uses: triage, space finder, document reading, 2 help assistants
Demo data	40 organisations · 93 users · 7 buildings · 123 spaces · six months of activity
Tests	281 back-end and 16 front-end unit tests · 42 live access probes · CI on every push
Deployment	4 containers (web, api, postgres, adminer), one command, one origin

PART 1

The big picture

What LeaseManager is, who uses it, and the chain it automates

1.1 What it is

LeaseManager is a web platform for leasing office space in Qatar. **Property companies** (the landlords) list offices, desks and meeting rooms in their buildings. **Companies looking for space** (the renters) find them on a public map, apply, visit, sign a lease, book rooms, pay rent and report problems. Everything happens in one application, and amounts are in Qatari riyals (QAR).

The difficulty — and the core of the project — is that **several competing property companies share one installation**. Each must see its own buildings, renters, invoices and tickets, and never a single row, total or counter belonging to a competitor. A renter company is itself an organisation, with its own administrator and employees. That makes three levels of organisation, nested.

1.2 Three levels, nine roles

Level	Role in the code	In the demo	What they do	Home page
1 · Platform	SUPER_ADMIN	Youssef Mansour	Runs the platform; the **only** role that reads across organisations	`/admin/dashboard`
2 · Landlord	CLIENT_ADMIN	Khalid Al-Mansouri	Owns the company workspace: team, company profile, and everything a manager does	`/admin/site-dashboard`
2 · Landlord	MANAGER	Fatima Al-Kuwari	Buildings, spaces, applications, bookings, contracts, maintenance	`/admin/site-dashboard`
2 · Landlord	FINANCE	Omar Haddad	Invoices, payments to confirm, revenue, exports	`/admin/finance-dashboard`
2 · Landlord	RECEPTIONIST	Maria Santos	Confirmation calls, visits, documents	`/admin/reception`
2 · Landlord	MAINTENANCE	Rajesh Nair	Takes, works and resolves maintenance tickets	`/admin/maintenance-dashboard`
3 · Renter	TENANT_ADMIN	Noor Al-Hajri (Acme Corp)	The renter company's admin: bookings, contracts, billing, team	`/portal/dashboard`
3 · Renter	TENANT_EMPLOYEE	Karim Saad (Acme Corp)	Books rooms, reports problems; sees only their own bookings	`/portal/dashboard`
—	GUEST	any visitor	Public map, AI space finder, application form — no account	`/map`

In the code an organisation is called a **tenant** (software meaning: one customer of the platform). The field `tenants.organization_type` tells them apart: `CLIENT` is a property company, `RENTER` is a renter company. The roles are in `backend/src/constants/enums.ts`.

1.3 The chain the platform automates



- **Public map** — a visitor browses published spaces, or asks the AI space finder in plain words.
- **Application** — the visitor applies for a space; the property manager accepts or refuses.
- **Intake** — the in-person part of the market: reception calls the applicant, a visit happens, documents are collected, and the manager finalizes. Each step is a booking state.
- **Lease** — a contract with a monthly rent; the space becomes reserved, then occupied.
- **Invoice** and **payment** — monthly rent and one-off charges in QAR; payments by cash, bank transfer or cheque, confirmed by finance.
- Alongside: **maintenance** (tickets with AI triage), **live notifications**, the **audit log** and **analytics** for every role.

1.4 The demo world

The command `npm run db:seed:full` (in `backend/`) wipes the database and builds a realistic Doha in about ten seconds:

	Content
Property companies	**Msheireb Properties** (3 buildings, 52 spaces) · **West Bay Offices** (2 buildings, 32 spaces) · **Lusail Business Hub** (2 buildings, 39 spaces) — 123 spaces in 7 buildings, with real coordinates
Renter companies	20 (Acme Corp, Qatar Fintech, Doha Design Studio…), each with an admin and employees; plus applicants in the intake pipeline and prospects
Activity	About 680 bookings, 490 invoices, 440 payments, 102 maintenance tickets, 450 notifications, 800 audit entries — six months of history, with dates relative to the day you seed
Accounts	93 users, all with the password `Password123!` — e.g. `manager@msheireb.test`, `tenant.admin@acme-corp.test`

> **In one sentence — if you can say this, you can explain this part**
> *LeaseManager lets several competing property companies lease office space to other companies on one platform — from the public map to the collected rent — while keeping every organisation's data strictly its own.*

PART 2

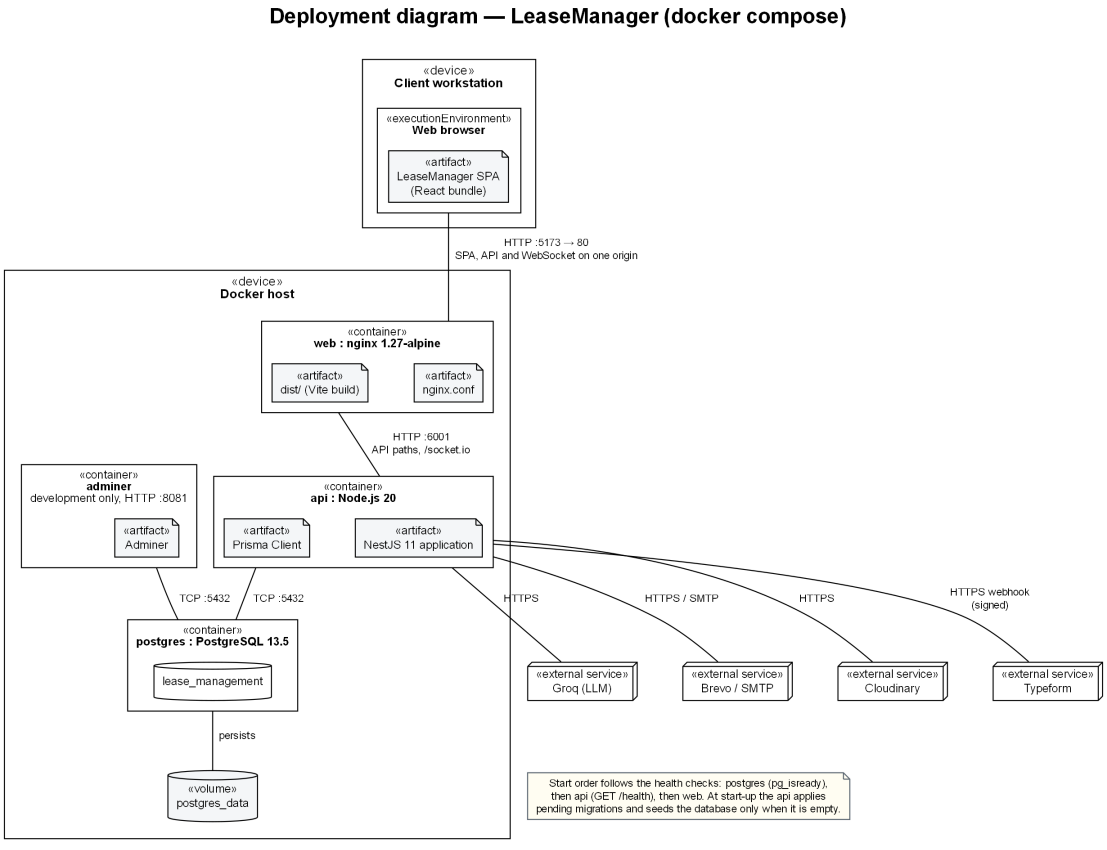
Architecture

Four containers, one origin, a NestJS API and a React application

2.1 The four containers

The whole platform starts with one command, `docker compose up -d`, from the repository root (`docker-compose.yaml`). It runs four containers:

Container	Technology	Address on your machine	Role
web	nginx 1.27 + the React build	`http://localhost:5173`	Serves the application and forwards API calls to the api container
api	NestJS 11 on Node 20	`http://localhost:6001`	All business rules: REST API, WebSocket gateway, scheduled jobs; Swagger at `/api`
postgres	PostgreSQL 13.5	`localhost:5433`	The data: 25 tables, created by 36 Prisma migrations
adminer	Adminer	`http://localhost:8081`	A database viewer, for development only



Deployment diagram (docs/diagrams/17-deployment-diagram.png)

One origin

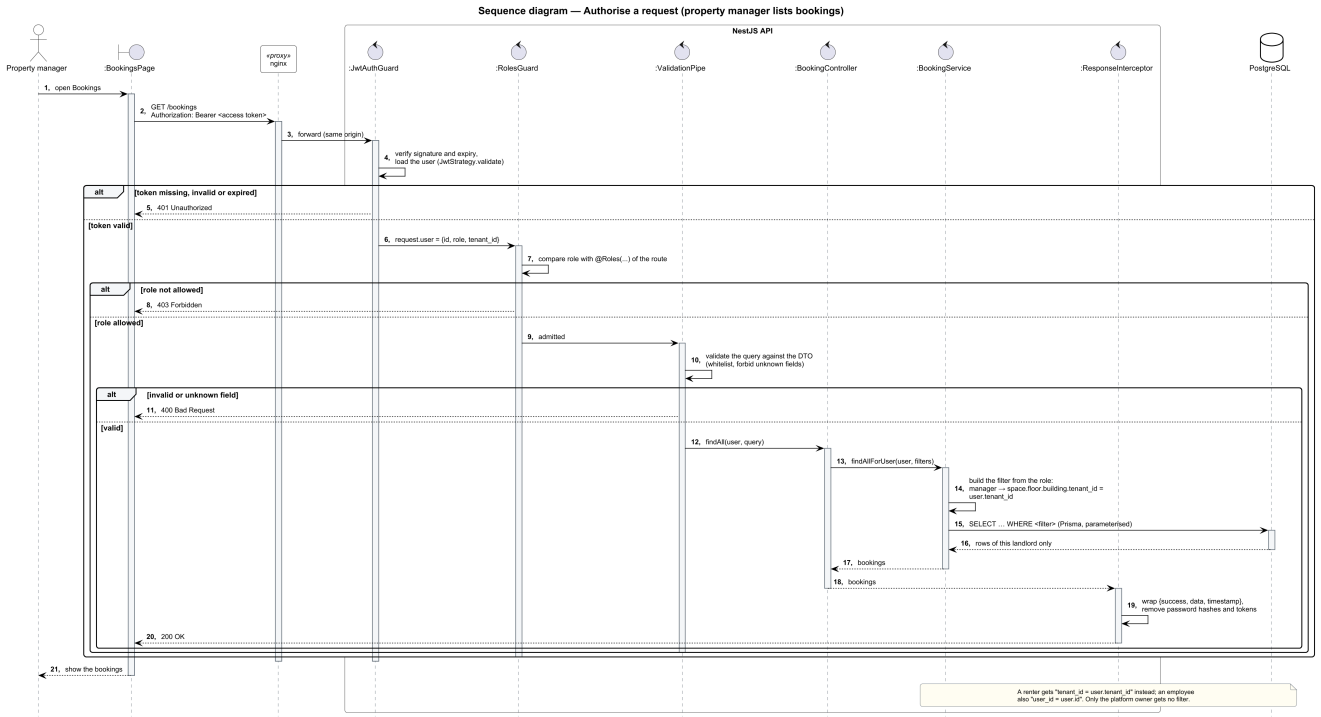
The browser only ever talks to `localhost:5173`. nginx (`frontend/nginx.conf`) looks at each path: API prefixes such as `/auth`, `/bookings` or `/billing` go to the api container; everything else is the React application. Because the page and the API share one origin, there is no CORS to configure and no server address is baked into the JavaScript bundle. One special case: `/spaces` is both an API path and a public page, so nginx sends a browser *page load* (it asks for HTML) to the application and an API call (it asks for JSON) to the API.

2.2 The life of one request

Follow one request: Fatima, the Msheireb property manager, opens the bookings list.

1	**The page asks** — React calls `bookingApi.getAll()` (`frontend/src/api/services.ts`). The axios client adds the header `Authorization: Bearer <access token>`, read from the browser tab's sessionStorage.
2	**nginx forwards** — `GET /bookings` matches an API prefix, so nginx passes it to `api:6001`.
3	**Middleware** — compression, security headers (Helmet), the CORS check, and the rate limiter when it is enabled (`backend/src/main.ts`).
4	**Who are you?** — `JwtAuthGuard` checks the token's signature and expiry; `JwtStrategy.validate` loads the user. A bad or expired token gets 401.
5	**Are you allowed here?** — `RolesGuard` compares the user's role with the route's `@Roles(...)` list. Not on it: 403.
6	**Is the input valid?** — the global `ValidationPipe` checks the query and body against the DTO class; any unknown field is rejected with 400.
7	**What may you see?** — the controller calls `findAllForUser(user, …)` in the service, which builds the database filter from the user: a landlord gets the bookings in its buildings, a renter its own, an employee only the ones they made.
8	**The database** — Prisma turns the filter into a parameterised SQL query for PostgreSQL.
9	**The answer** — `ResponseInterceptor` wraps the result as `{ success, data, … }` and strips any password hash or session token; `PerformanceInterceptor` logs the timing; errors go through `HttpExceptionFilter` so they all have the same JSON shape.
10	**Back in the browser** — the client unwraps `data`, `mapBookings` normalises the field names, TanStack Query caches the list, React renders it.

If the access token has expired, the API answers 401; the client then calls `POST /auth/refresh-token` **once** (parallel requests wait for the same refresh) and replays the request with the new token (`frontend/src/api/client.ts`).



Sequence diagram — authorising a request (docs/diagrams/05-sequence-authorisation.png)

2.3 The back end: NestJS modules

NestJS organises code in **modules**. Each feature module has a **controller** (the HTTP routes, with decorators for guards and roles), a **service** (the business rules) and **DTOs** (classes that describe and validate the input). Services reach the database through the shared `PrismaService`. The project has 30 controllers exposing about 220 routes, all documented in Swagger.

Area	Modules (folders in backend/src)	What they cover
Identity and access	auth, user, tenant, security, common	Sign-in, sessions, roles, organisations; the guards, the access policy, interceptors and filters live in `common`
Portfolio	building, floor, space, addon-service, promotion-code	Buildings, floors, spaces and their map; extras; discount codes
Commercial	booking-application, booking, lease-contract, billing, forms	Applications, the booking workflow, contracts, invoices and payments; company applications from Typeform
Operations	maintenance, notification, websocket, audit, mail, brevo, email-sequence, upload	Tickets, live notifications, the audit log, email delivery, files
Insight	analytics, report, export, search	Dashboards and trends, saved reports, CSV/Excel exports, global search
AI	ai	The LLM client, maintenance triage, the space finder, document reading
Platform	prisma, config, logging, performance, monitoring, cache, tasks, marketplace, crisp	Database access, configuration, logs, timing, caching, scheduled jobs, listing sync, chat widget

2.4 The front end: React

- **Stack** — React 19, Vite 7 and TypeScript; Ant Design 6 for components, Recharts for charts, Leaflet for maps.
- **Routing** (`frontend/src/App.tsx`) — public pages (`/`, `/map`, `/login`, `/apply/:spaceId`…), the renter portal under `/portal/*` and the staff back office under `/admin/*`. Each page is wrapped in a `RoleRoute` listing the roles allowed in; anyone else is sent to their own home page.
- **Layout** (`layouts/MainLayout.tsx`) — the sidebar menu is built per role, so nobody is offered a page they cannot open.
- **Talking to the API** — `api/services.ts` groups the calls by domain (`bookingApi`, `billingApi`…); `api/client.ts` adds the token, unwraps responses and refreshes the session; identical requests already in flight are shared, not repeated.
- **Data on screen** — TanStack Query fetches, caches and refreshes server data; it does not retry a refused (4xx) request.
- **State** — Zustand stores: `authStore` (the user and tokens, in **sessionStorage**, so each browser tab holds its own session) and `themeStore` (light or dark).
- **Real time** — the notification bell keeps a Socket.IO connection to `/notifications`.

> **In one sentence — if you can say this, you can explain this part**
> *One command starts four containers; nginx serves the React application and forwards API paths to NestJS on the same origin, where every request passes the token check, the role check, input validation and a role-based data filter before Prisma touches PostgreSQL.*

PART 3

The data model

25 tables, one schema file, and the idea that one record has two owners

The whole database is described in one file, `backend/prisma/schema.prisma`. Prisma generates a typed client from it, and 36 migrations in `prisma/migrations/` create the 25 tables.

3.1 The 25 tables, by area

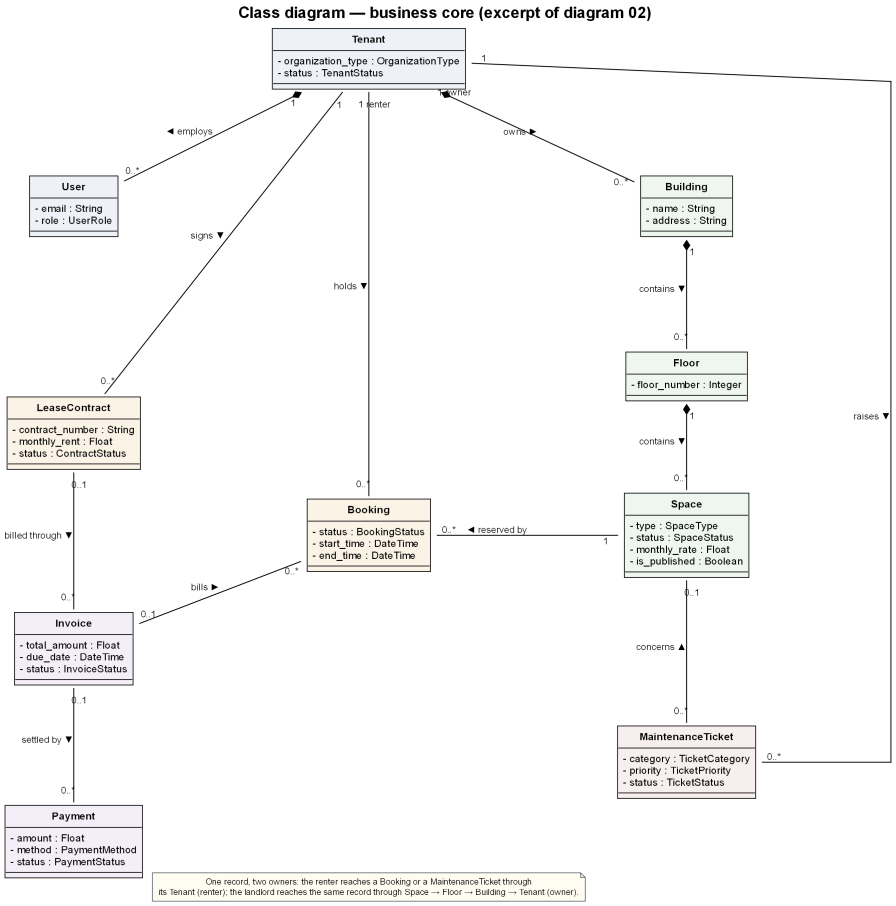
Area	Tables	What they hold
Organisations and people	Tenant, User, UserSession, TenantApplication, TenantOnboardingSequence	An organisation (landlord or renter) and its users; one session row per signed-in device; companies applying to rent; their welcome emails
Portfolio	Building, Floor, Space, SpaceFeature, AddOnService, SpaceAddOnService	The landlord's property: a building has floors, a floor has spaces (type, status, prices, map position, published or not); the extras a space offers
Commercial	BookingApplication, BookingApplicationAddOn, Booking, BookingAddOn, BookingDocument, LeaseContract, Invoice, InvoiceLine, Payment, PromotionCode	Applications from the map; bookings with their extras and scanned documents; leases; invoices with their lines; payments; discount codes
Operations and insight	MaintenanceTicket, Notification, AuditLog, Report	Tickets; notifications shown in the bell; the record of every privileged action; saved reports

3.2 One record, two owners

This is the single most important idea in the project. A booking belongs to **two** organisations, by two different paths:

- **The renter** owns it through `booking.tenant_id` — "I am the one who booked it."
- **The landlord** owns it through `booking → space → floor → building.tenant_id` — "those are my walls."

The same is true of maintenance tickets, invoices (reached by the landlord through their bookings) and lease contracts (reached through invoice → booking → space → building). Every query that serves a landlord must follow the building path; every query that serves a renter follows `tenant_id`. Mixing the two up is the easiest mistake to make and the hardest to see: with only one landlord in the data, "show everything" and "show my portfolio" return the same rows.



Class diagram — the business core (docs/diagrams/02p-class-core.png)

3.3 Every status, in one table

Thing	Values	Notes
Space status	AVAILABLE · OCCUPIED · RESERVED · MAINTENANCE · OUT_OF_SERVICE	Only AVAILABLE and published spaces reach the public map
Space type	DEDICATED_OFFICE · FLEXIBLE_DESK · HOT_DESK · MEETING_ROOM · CONFERENCE_ROOM · PHONE_BOOTH · EVENT_SPACE	Offices and desks are leased by the month; rooms and halls are booked by the hour or day
Booking (12)	DRAFT · PENDING_APPROVAL · PENDING_PHONE_CONFIRMATION · AWAITING_PHYSICAL_VISIT · DOCUMENTS_PENDING_UPLOAD · ACTIVE · CONFIRMED · CHECKED_IN · COMPLETED · CANCELLED · REFUSED · NO_SHOW	Three of them exist only for the in-person intake (part 5.3)
Booking application	PENDING · ACCEPTED · REFUSED	
Lease contract	DRAFT · ACTIVE · RENEWED · EXPIRED · TERMINATED	
Invoice	DRAFT · ISSUED · SENT · PARTIALLY_PAID · PAID · OVERDUE · CANCELLED	Types: MONTHLY_RENT, ONE_TIME_CHARGE, DEPOSIT, PENALTY, CREDIT
Payment	PENDING · COMPLETED · FAILED · REFUNDED	Methods: CASH, BANK_TRANSFER, CHECK (and card or online values for payments taken elsewhere)
Maintenance ticket	OPEN · ASSIGNED · IN_PROGRESS · RESOLVED · CLOSED · CANCELLED	Priority LOW to EMERGENCY; category PLUMBING, ELECTRICAL, HVAC, CLEANING, FURNITURE, IT_EQUIPMENT, OTHER
Organisation	TRIAL · ACTIVE · SUSPENDED · CLOSED · PENDING	A renter starts in TRIAL when its application is approved

In one sentence — if you can say this, you can explain this part

Twenty-five tables in one Prisma schema; a booking, a ticket, an invoice or a lease belongs to the renter through its tenant_id and to the landlord through the building it sits in — and every query must choose the right path.

PART 4

Security and isolation

How a user is recognised, what they may do, and how competitors stay apart

4.1 Signing in and staying signed in

1	**Login** — `POST /auth/login` with email and password. The password is checked with bcrypt against the stored hash (`auth.service.ts`). An unknown email and a wrong password get the same 401 answer, so the form cannot reveal which addresses have an account; a wrong password is written to the audit log as a warning.
2	**A session** — a row is created in `user_sessions` — one per signed-in device — and two tokens are signed: an **access token** (JWT, valid `JWT_EXP_IN` = 1 day here) and a **refresh token** (7 days).
3	**Every request** — the access token travels in the `Authorization` header; the server verifies it and loads the user.
4	**Refresh** — `POST /auth/refresh-token` checks the refresh token against its session row, then issues a new pair and **rotates** the stored token, so an old refresh token cannot be replayed.
5	**Logout** — deletes the session row; `/auth/sessions` lets a user see and revoke their other devices.
6	**New accounts** — nobody is ever shown a temporary password: a new admin or employee gets an email link to set their own (`requestPasswordReset`).

In the browser the tokens live in **sessionStorage**, which belongs to one tab. That is why you can be Fatima in one tab and Noor in another at the same time, and why closing the tab signs you out.

4.2 The five checks on every request

#	Check	Where	Stops
1	Token	`JwtAuthGuard` + `JwtStrategy`	Anyone not signed in (401)
2	Role	`RolesGuard` + `@Roles(...)` on the route	A signed-in user calling a route their role may not use (403)
3	Input	global `ValidationPipe` (whitelist, forbid unknown fields)	Malformed input and extra fields such as a forged `role` (400)
4	Ownership	the service's `…ForUser` methods	Reading or changing a record of another organisation (403 or 404)
5	Data filter	`AccessPolicyService` and per-service filters	Lists and totals that would include other organisations' rows

Check 5 is where isolation really lives. For every list, the service turns the user into a Prisma `where`. `isCrossTenantReader` is true for the platform owner only; for everyone else the filter names their organisation — through `tenant_id` for a renter, through the building for a landlord:

List	Landlord staff see…	Renter sees…
Spaces	spaces whose floor's building is theirs	available spaces (any landlord) — that is how they book
Bookings	bookings on spaces in their buildings	their organisation's bookings; an employee only their own
Invoices	invoices linked to bookings in their buildings	their own invoices
Contracts	contracts whose invoices reach their buildings	their own contracts
Tickets	tickets on spaces in their buildings	their own tickets
Analytics	the same building paths, for every chart and total	their own figures

4.3 The other protections

- **Security headers** (Helmet) and a **CORS** allow-list (`main.ts`).
- **Rate limiting** — a global limit per IP address (on in production; in development only with `RATE_LIMIT_ENABLED=true`), plus tighter limits on single routes with `@RateLimit`, always on: 5 guest applications per minute per IP, 30 to 50 searches per minute.
- **No secrets in responses** — the response interceptor removes password hashes and refresh tokens from every user object it sends (`common/interceptors/response.interceptor.ts`).
- **Injection** — every query goes through Prisma, which parameterises it.
- **Live notifications** — the Socket.IO gateway verifies the token on connection and only lets a client into its own user, organisation and role rooms (`websocket/notifications.gateway.ts`).
- **Webhooks** — a Typeform submission is accepted only with a valid signature.
- **Audit log** — privileged actions are recorded with the actor, organisation, IP address and a severity (INFO to CRITICAL).
- **Uploads** — ownership is checked before a file is written (`upload.controller.ts`); files go to Cloudinary when it is configured, otherwise (or if Cloudinary fails) to local storage. A document sent to the AI reader is held in memory for the request only and never stored.

4.4 How the isolation is proved

- **The access audit** — `npm run audit:access` (in `backend/`) signs in as real users of competing companies and fires **42 probes** in 7 groups at the running server: reads limited to your own organisation, forged organisation ids refused, out-of-reach records, writes against another organisation, each landlord seeing only its own portfolio, the public space finder unable to reach past the map, and self-promotion to platform owner refused. All 42 pass.
- **The arithmetic check** — each landlord's totals (spaces 52 + 32 + 39, tickets 48 + 30 + 24, revenue) add up *exactly* to the platform's totals. If anything leaked or was counted twice, the sum would not close.
- **Unit tests** — the filters themselves are tested (`access-policy.service.spec.ts`, `analytics.service.spec.ts`, `export.service.spec.ts`…).

> **The lesson of the project**
> *None of the cross-tenant defects found during development came from a broken security function. Every one came from a call site that forgot to use it — and several stayed invisible while the test data held a single landlord. The shape of the test data is part of the security surface.*

> **In one sentence — if you can say this, you can explain this part**
> *Every request is checked five times — token, role, input, ownership and a data filter built from the user — and the isolation is proved by 42 live probes and by totals that add up exactly across the three competing landlords.*

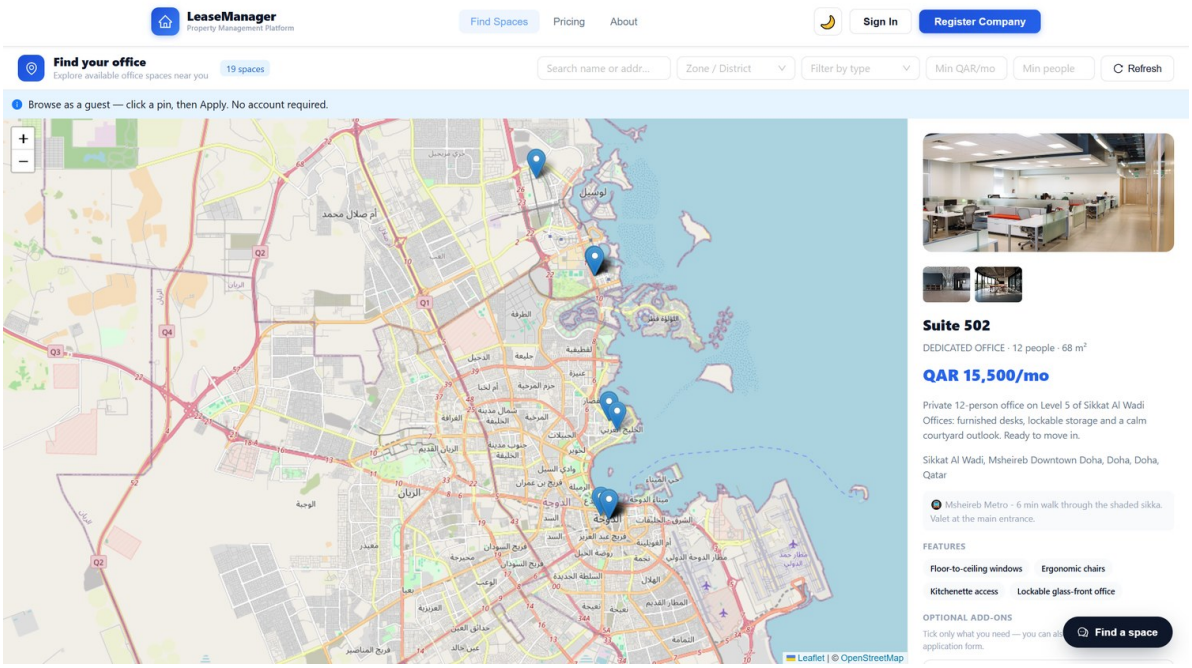
PART 5

The business flows

Every feature as it actually runs, step by step

5.1 A visitor finds a space

1	**The map** — `/map` loads `GET /spaces/public/map`: only spaces that are published, available and placed on the map. No account is needed. Clicking a pin opens the space: photos, monthly rent in QAR, features, transport notes.
2	**Or ask** — the **space finder** (`POST /public/space-finder`) takes a request in plain words, such as “an office in Msheireb for 8 people”, and answers with real listings (part 6.2).
3	**Apply** — `/apply/:spaceId` sends `POST /booking-applications/guest`: who is applying (a person or a company), contact details, start date, duration, headcount, optional add-ons. The application is stored as PENDING and the landlord's managers are notified.



The public map with a space open

5.2 A company joins the platform

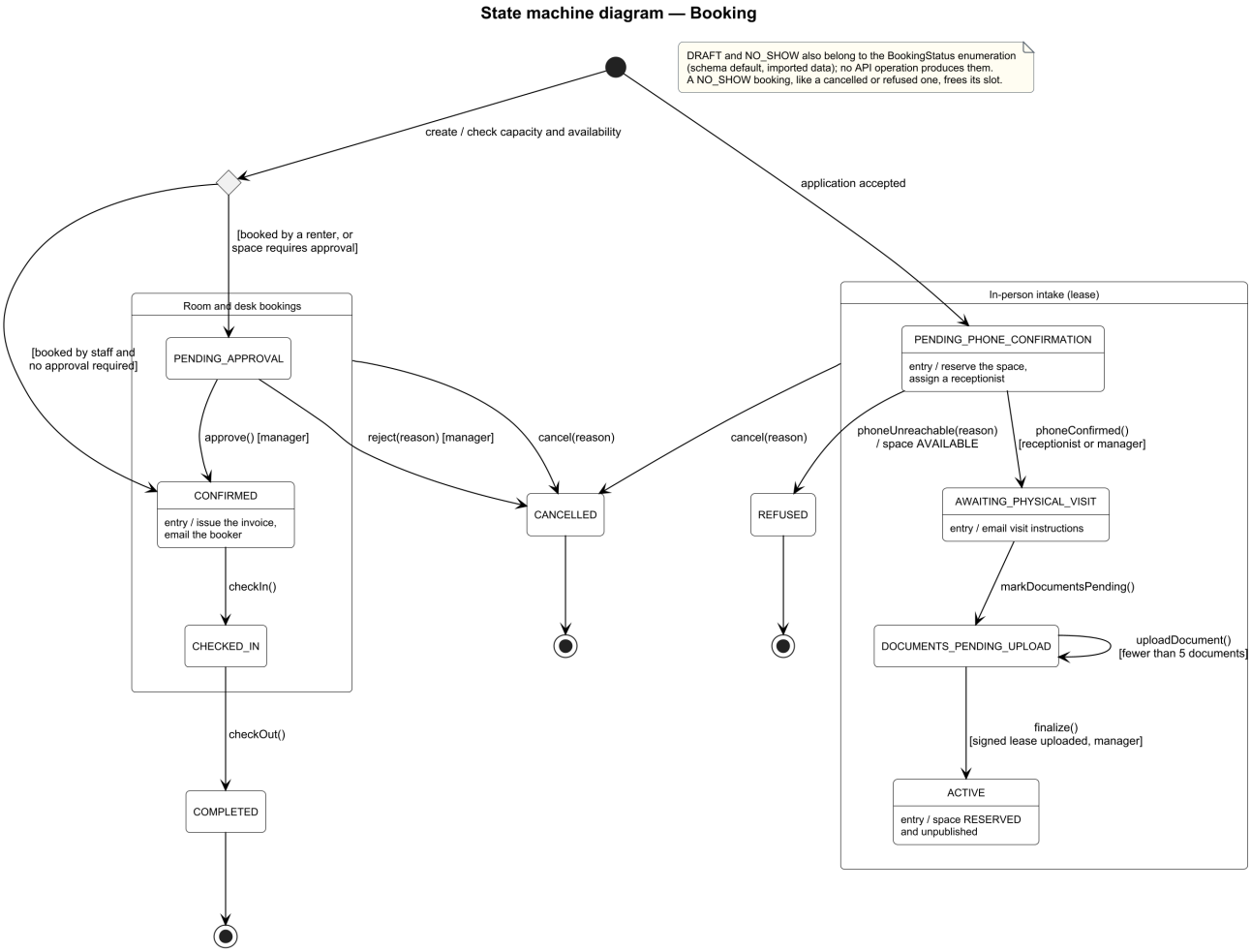
Companies can also apply through a **Typeform** questionnaire. Typeform calls `POST /webhooks/typeform`; the request is accepted only with a valid signature.

1	**Arrive** — the submission creates the company as a RENTER organisation in **PENDING** status, and a **tenant application** addressed to the property company it applied to (`forms/typeform-webhook.service.ts`).
2	**Review** — that company's manager sees it under Tenant Applications, and approves it or rejects it (with an optional reason).
3	**Approve** — one database transaction moves the organisation to **TRIAL**, marks the application APPROVED and creates the company's TENANT_ADMIN user, whose stored password is a random hash nobody knows (`forms/tenant-application.service.ts`).
4	**First sign-in** — the new admin receives an email link to choose a password — no temporary password is ever shown or sent.

5.3 The in-person intake: from application to lease

This is the part of the market that existing products ignore: in Qatar a lease is concluded face to face. The platform turns each step into a booking state, and has a receptionist role to carry it.

1	**Accept** — the manager accepts a booking application (`PATCH /booking-applications/:id/accept`). In one transaction the platform creates the applicant's organisation and account if needed, creates a booking in **PENDING_PHONE_CONFIRMATION**, assigns a receptionist of that landlord, and marks the space **RESERVED**. Refusing instead records a reason.
2	**Call** — reception calls the applicant. Reached: `phone-confirmed` → **AWAITING_PHYSICAL_VISIT**, and the applicant is emailed visit instructions. Unreachable: the booking becomes **REFUSED** and the space AVAILABLE again.
3	**Visit** — after the visit, `mark-documents-pending` → **DOCUMENTS_PENDING_UPLOAD**.
4	**Documents** — reception or a manager attaches the scans to the booking, at most 5: copy of the commercial registration (CR), copy of the Qatar ID (QID), trade licence, proof of payment — and the **signed lease**, which is mandatory.
5	**Finalize** — the manager finalizes (`PATCH /bookings/:id/finalize`): the booking becomes **ACTIVE**, the space RESERVED and unpublished, and the renter is emailed.



State machine diagram — Booking (docs/diagrams/13-state-booking.png)

5.4 Booking a room

Renters book meeting rooms, conference rooms, desks and event halls themselves — from the Floor Map, the calendar or the bookings page.

1	**Create** — `POST /bookings`: the server checks the space exists, the head count fits its capacity, and the time slot does not overlap another booking (`checkAvailability`) **before** writing. A renter's booking starts in **PENDING_APPROVAL**; staff bookings are confirmed directly unless the space requires approval.
2	**Approve** — the landlord's manager approves (`PATCH /bookings/:id/approve`) → **CONFIRMED**: the renter is emailed and notified, and an invoice is issued automatically. Rejecting records a reason.
3	**Use** — check-in → **CHECKED_IN**, check-out → **COMPLETED**. A booking can also be cancelled. Every booking blocks its slot except cancelled, refused and no-show ones.

5.5 Leases

A lease contract (`lease-contract` module) has a number such as `CT-202510-92650B86`, dates and a monthly rent. It moves from **DRAFT** to **ACTIVE**, and ends as **EXPIRED**, **TERMINATED** or **RENEWED**. A contract can be generated from a confirmed booking; signing a DRAFT makes it ACTIVE. When a company signs its **first** lease, an onboarding email sequence starts, and a job at 09:00 each morning sends the next email (`email-sequence`). Every morning at 08:00 another job (`tasks/scheduled.service.ts`) marks leases past their end date EXPIRED and reminds renters of leases ending in 60, 30 and 7 days; the renewals page lists them for staff.

5.6 Billing and payments

- **Invoices** have lines, a total in QAR and a type: monthly rent, one-off charge (rooms, events, extras), deposit, penalty or credit. Promotion codes give a percentage or a fixed discount; codes are unique per landlord, so two landlords can both run SUMMER25.
- **No payment gateway.** In this market rent is paid by cash, bank transfer or cheque, so the platform *records* payments rather than processing them.
- **A renter reports a payment** → it is stored as **PENDING** (a cheque can carry a scanned copy).
- **Finance confirms it** (`PATCH /billing/payments/:id/confirm`) → **COMPLETED**, with the name of who confirmed. The invoice then updates itself: fully covered becomes **PAID**, partly covered **PARTIALLY_PAID**.
- **Overdue** — at 08:00 each morning the job marks unpaid invoices past their due date **OVERDUE**, then emails reminders 1, 7 and 14 days after the due date.
- **Refunds** mark a payment REFUNDED and keep the history.

5.7 Maintenance

1	**Report** — `POST /maintenance`. A renter can only report on spaces it has booked or leased — an isolation rule hidden inside a feature.
2	**Triage** — while the report is typed, the AI suggests a category, a priority and a technician (part 6.1).
3	**Assign or accept** — a manager assigns a technician (→ ASSIGNED), or a technician takes a ticket from the open queue with **Accept**; once taken, another technician cannot claim it.
4	**Work** — start → **IN_PROGRESS**, resolve (with an optional cost) → **RESOLVED**, close → **CLOSED**. Each change notifies the people concerned.

5.8 Notifications

A notification is first saved in the database, then pushed live over Socket.IO to the namespace `/notifications`. When the bell connects, the gateway verifies the token and places the client in three rooms: its user, its organisation and its role, so a message can target one person, a company or every manager. Emails go out through Brevo, or SMTP, or are logged to the console when neither is configured — the platform never fails because email is down.

5.9 Dashboards, analytics and exports

- **One dashboard per role**: platform owner, property manager, finance, technician, reception, renter.
- **Analytics** (`analytics.service.ts`): revenue trend, bookings by status, top spaces, revenue by renter, all scoped as in part 4.
- **Revenue forecast**: for each of the next 12 months, the rent of the active and renewed leases running that month and the rent that expires that month; plus the leases ending within 120 days.
- **Occupancy heatmap**: for each space of a building, the share of bookable hours (08:00–22:00 Doha time, every day) that was booked (`utils/working-hours.ts`).
- **Predictive maintenance**: a risk score from 0 to 100 for each open ticket — 20 to start, +12 for each other ticket on the same space in the last 120 days (at most +40), +25 if urgent or emergency, +15 above 4,000 usage hours, +5 per year of equipment age (at most +25). 70 and above is *high*, 45 and above *watch*. A transparent rule, not a trained model.
- **Exports**: bookings, invoices, payments, tickets, spaces and organisations as CSV or Excel, limited to the caller's scope.
- **Audit log** and **global search** for the platform owner and staff.

> **In one sentence — if you can say this, you can explain this part**
>
> *A visitor finds a space and applies; the manager accepts; reception calls and receives the visit; the signed lease is uploaded and the booking becomes active; then rent is invoiced in QAR, the renter reports a payment, finance confirms it, and maintenance runs alongside with AI triage.*

PART 6

The AI features

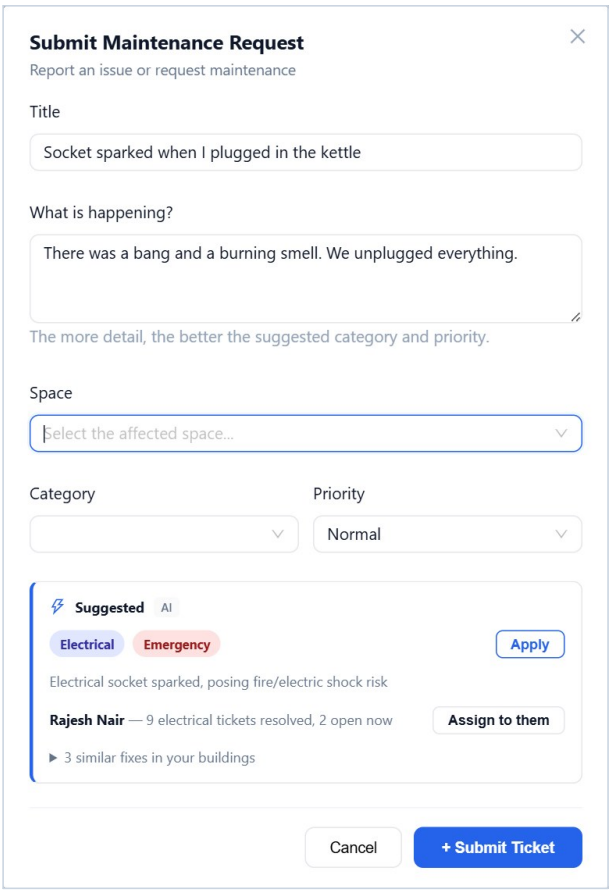
The model proposes; the application decides

All AI calls go through one client, `ai/openai.service.ts`. It speaks the OpenAI API, so the provider is a setting: `OPENAI_BASE_URL` points at **Groq** and `OPENAI_MODEL` is `openai/gpt-oss-120b`. Changing provider is configuration, not code.

Without an API key, nothing in the business stops: triage falls back to its keyword classifier, the space finder still searches (using the district named in the visitor's own words) and answers with a fixed sentence, document reading returns no fields, and the two help assistants say they are unavailable.

6.1 Maintenance triage

1	**Classify** — the title and description go to the model, which must answer with a category and a priority (`maintenance-triage.service.ts`).
2	**Validate** — if either value is not in the application's own list, the whole answer is discarded and a **keyword classifier** decides instead. The suggestion shows its source: "AI" or "keyword match".
3	**Name a technician — without the model** — for staff who can assign, the technician comes from ticket history: among the landlord's active technicians, score = 10 × tickets they resolved in that category − 6 × tickets they have open; a tie goes to the lighter load. The reason is shown, e.g. "Rajesh Nair — 9 electrical tickets resolved, 2 open now".
4	**Similar fixes** — the three most recent resolved or closed tickets of the same category in the landlord's buildings are listed for reference.
5	**Apply** — the manager applies the suggestion or ignores it; submitting a ticket never depends on the model.



Triage of a new ticket, as the Msheireb manager sees it

Reading the suggestion

Suggested · AI — where the answer came from. It reads "keyword match" when the model was not used or its answer was refused.

Electrical · Emergency — category and priority, proposed by the model and accepted because both are in the application's own lists. **Apply** copies them into the form.

Rajesh Nair — 9 electrical tickets resolved, 2 open now — the technician, chosen from ticket history by the score above, never by the model. **Assign to them** uses it.

3 similar fixes in your buildings — past tickets of the same kind, from this landlord's buildings only.

Why the technician is not the model's choice: a model asked to name a person invents people who do not work here, and a manager has to be able to defend the choice. "9 tickets resolved, 2 open" is a reason.

6.2 The space finder

A public, unauthenticated chat on the map (`ai/space-finder.service.ts`). It works in three stages, and the middle one is the point:

1	**Understand** — the model turns the conversation into criteria: space type, head count, monthly budget, area (a city, district or building such as Msheireb) and up to four keywords such as “parking”. A type that does not exist or a number that is not a positive figure is dropped; if the model names no area, the visitor's own words are searched for known districts.
2	**Search — without the model** — a plain Prisma query, always limited to the public filter (published, available, on the map), applies the criteria. If nothing matches, it drops the budget and size; if still nothing, the area too — and the reply says so.
3	**Reply** — the model writes at most three sentences about the rows the query returned. Its instructions forbid mentioning any other space, price or feature.

This is what makes it safe to expose: a visitor can write instructions aimed at the model and make it *say* anything, but cannot make it *read* anything beyond the public listings, because the model never selects data. The access audit tries four prompt-injection attacks; all fail.

6.3 Reading documents

`POST /ai/documents/extract` reads an uploaded PDF — a trade licence, a commercial registration, a Qatar ID or a cheque — and returns the fields a manager would otherwise retype, with a confidence and the list of fields it could not find. The text is taken from the PDF locally (`pdf-parse`), and only that text goes to the model. A scan or a phone photo has no text layer, so it is reported as unreadable rather than guessed; reading it would need a vision model. The server side is done; no screen calls it yet.

6.4 Two help assistants

- **Lease questions** — a card under the contracts list for staff (`POST /ai/lease-assistant`): plain-language answers about leases, told never to invent legal facts and to recommend checking with counsel.
- **Renter help** — a chat button on every page of the renter portal (`POST /ai/tenant-assistant`): how to book, pay, report a problem; for anything about policy it sends the renter to their property manager.
- Both are general help. The model receives the conversation and the user's role — never a row from the database — so there is nothing it could leak.

> **The principle, in the project's words**
> *The model proposes, it does not decide. Any value outside the application's own lists is discarded and a rule takes over; people are chosen from history, not by the model; and the model only ever sees data the query already returned.*

> **In one sentence — if you can say this, you can explain this part**
> *One configurable AI client serves ticket triage, the public space finder, document reading and two help assistants; the model only proposes — its answers are checked against the application's lists, people are chosen from history, and it never selects data itself.*

PART 7

Running, testing, deploying

Commands, configuration, the demo data, tests and CI

7.1 Starting the platform

Task	Command (PowerShell, from the repository root)
Start everything	`docker compose up -d` — add `--build` after changing code
Status	`docker compose ps` — api, postgres and web should all say "healthy"
Fresh demo data	`cd backend` then `npm run db:seed:full` (about 10 s)
API logs	`docker compose logs api --tail 50`
Access audit	`cd backend` then `npm run audit:access` (42 probes against the running server)
Tests	`cd backend; npx jest` and `cd frontend; npx vitest run`
Dev servers instead	`docker compose stop api web`, then `npm run start:dev` in `backend/` (port 6001) and `npm run dev` in `frontend/` (port 5173; Vite forwards the API paths to 6001)

Address	What it is
`http://localhost:5173`	The application (public home page)
`http://localhost:5173/map`	Public map, no account
`http://localhost:5173/login`	Sign-in — every demo account uses `Password123!`
`http://localhost:5173/api`	Swagger: every API route, with a "Try it out" button
`http://localhost:6001/health`	API health check
`http://localhost:8081`	Adminer — server `postgres`, database `lease_management`

7.2 How the containers are put together

- **api** is built from `backend/Dockerfile.dev`: it installs dependencies, generates the Prisma client, then at start-up applies pending migrations (`prisma migrate deploy`), seeds only if the database is empty, and runs NestJS in watch mode. The source folder is mounted, so code changes reload.
- **web** is built in two stages from `frontend/Dockerfile`: a Node stage runs `npm run build` (type check + Vite), then an nginx stage serves the result with `frontend/nginx.conf`.
- **postgres** keeps its data in a named volume, so it survives restarts.
- Configuration comes from `backend/.env`: database URL, JWT secrets and lifetimes, AI provider and model, Brevo/SMTP, Cloudinary, Typeform secret, rate limiting. Secrets never go in the code.

7.3 The demo data generator

`backend/prisma/seed-full.ts` builds the whole demo world described in part 1.4. Two design choices make it reliable: it is **deterministic** (a seeded random generator, so the same names and numbers come back every time) and its dates are **relative to today** (so "this week", due dates and expiring leases always make sense). Certain scenes are pinned for the demo: Acme's Conference 7 request waiting for approval, Wakra Coffee Roasters waiting for a call, Rumaila Consulting Group ready to finalize, Brightpath's QAR 29,600 cheque waiting for finance, the emergency leak in the Zubarah Room.

7.4 Tests and continuous integration

Kind	Tool	Count	What it covers
Server unit tests	Jest	281 in 23 suites	Access policy, analytics scoping, bookings, billing, contracts, exports, AI (classifier, space finder), auth, interceptors…
Client unit tests	Vitest	16	The API client (unwrapping, token refresh) and the month bucketing behind the charts
Live access audit	Node script	42 probes	Cross-organisation reads and writes, forged ids, space finder injection, privilege escalation
Type checking	TypeScript	both projects	Clean; part of the web build

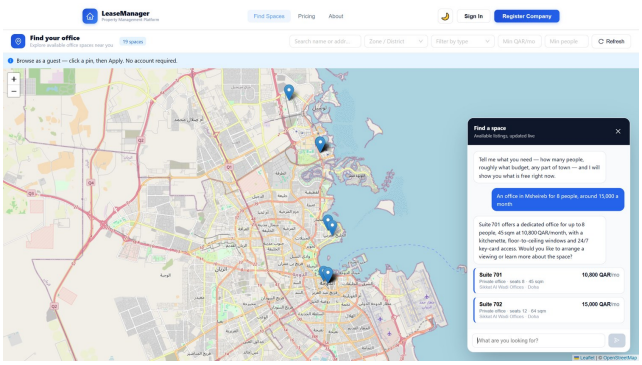
GitHub Actions (`.github/workflows/ci.yml`) runs on every push to main and every pull request: back end — lint, tests with coverage, build; front end — lint, type check, tests with coverage, build; plus a dependency security audit.

> **In one sentence — if you can say this, you can explain this part**
>
> *Docker Compose starts the stack with one command; a deterministic seed builds a realistic demo dated relative to today; 297 unit tests, 42 live access probes and a CI pipeline on every push keep it honest.*

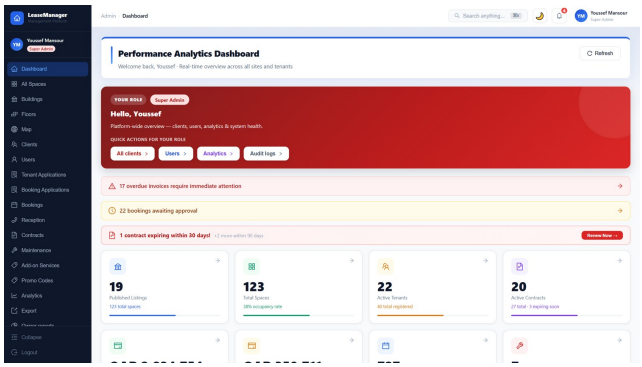
PART 8

A tour of the screens

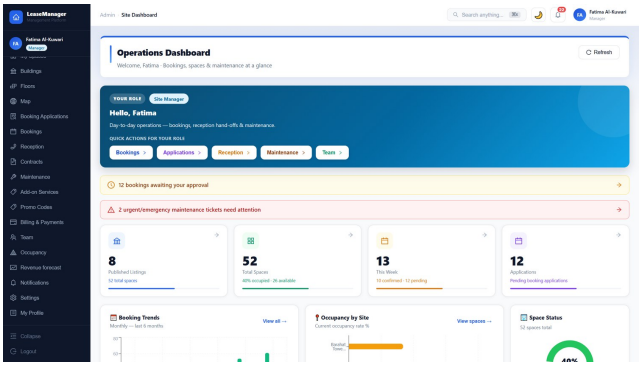
What each role sees — captured from the running application



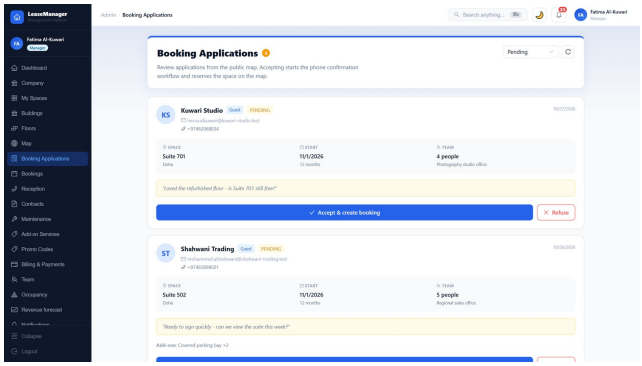
Visitor — the space finder answering “an office in Msheireb for 8 people” with real listings.



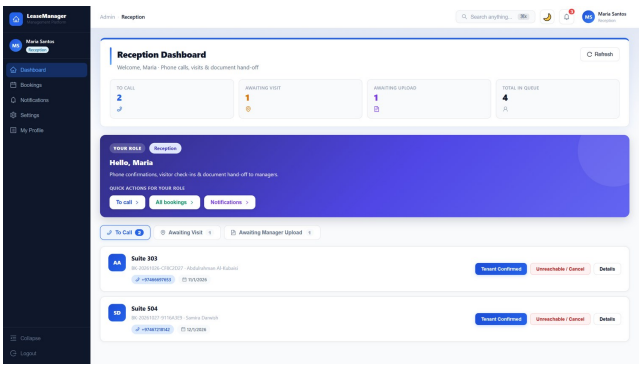
Platform owner — every organisation: 123 spaces, 22 active organisations, renewals.



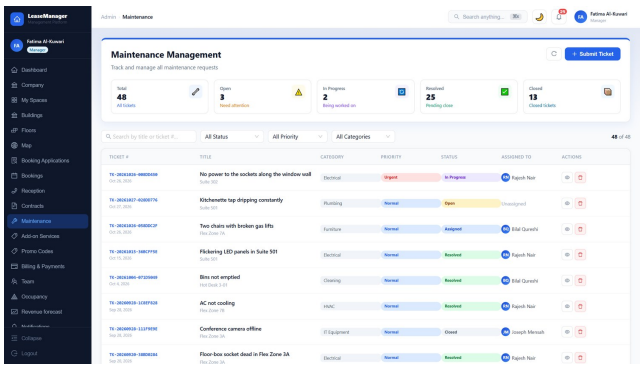
Property manager — the same screens, scoped to Msheireb: 52 spaces, occupancy per building.



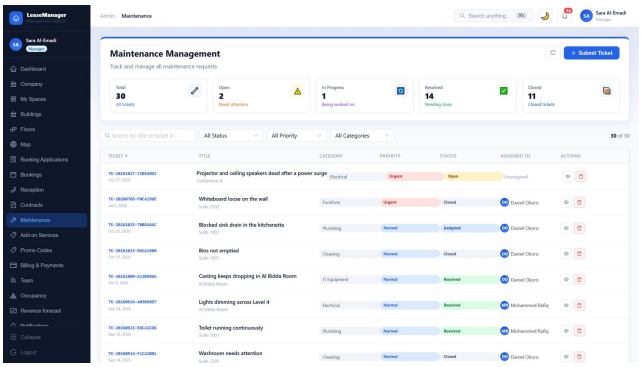
Property manager — applications from the public map, to accept or refuse.



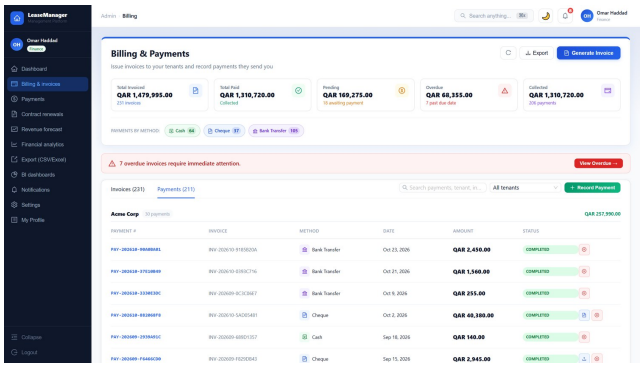
Reception — who to call, who is awaiting a visit, whose documents are missing.



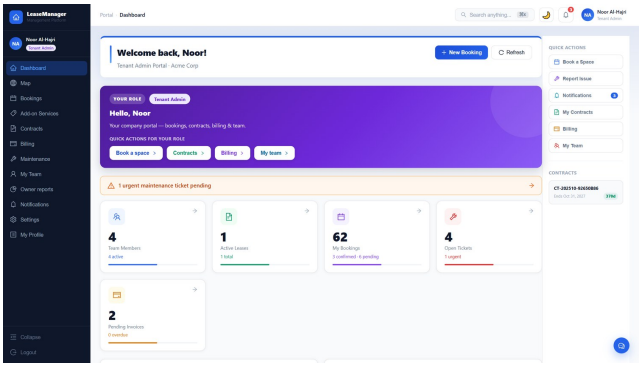
Msheireb's manager — 48 maintenance tickets…



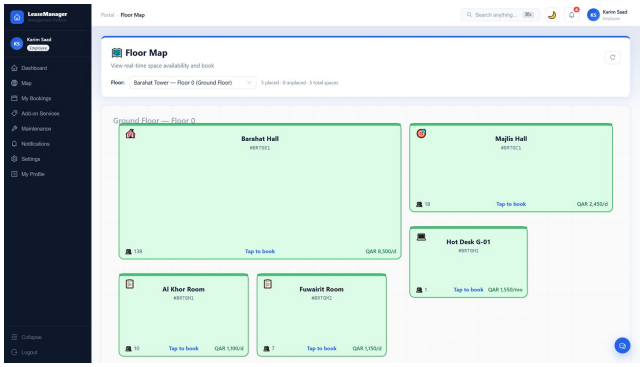
West Bay's manager — …and on the same screen, 30 different ones. No overlap.



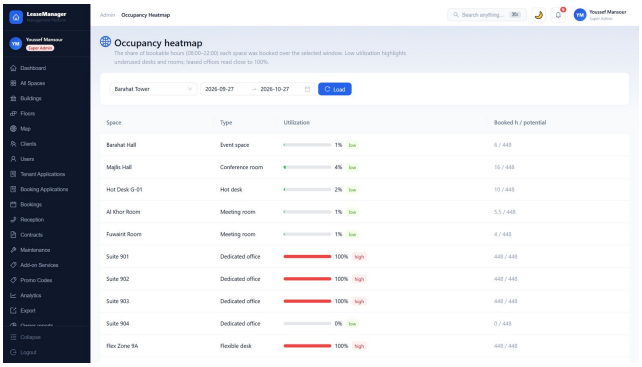
Finance — payments by method, to confirm; amounts in QAR.



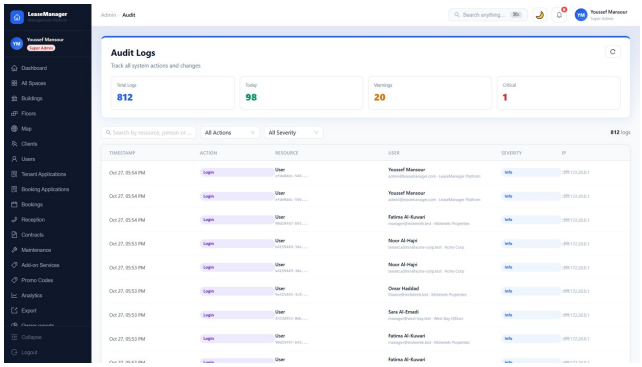
Renter admin — Acme Corp's own portal: bookings, lease, invoices, team.



Renter employee — the floor map: available rooms with capacity, price and “Tap to book”.



Analytics — the occupancy heatmap of Barahat Tower.



Audit trail — who did what, from which organisation and IP address.

PART 9

Questions for the defense

What a jury tends to ask, and answers grounded in the code

What stops one company from seeing another's data?

Five checks on every request: the token (JwtAuthGuard), the role (RolesGuard and @Roles), input validation that rejects unknown fields, an ownership check in the service, and a data filter built from the user by the access policy. Only the platform owner reads across organisations. A landlord reaches records through its buildings, a renter through tenant_id.

How do you know the isolation works?

By measurement. npm run audit:access fires 42 probes at the running server as users of competing companies; all pass. And each landlord's totals add up exactly to the platform totals — 52 + 32 + 39 spaces, 48 + 30 + 24 tickets — so nothing leaks and nothing is counted twice. 297 unit tests cover the filters themselves.

Why one shared database rather than one database per company?

Row-level isolation keeps one schema, one migration chain and one deployment, and the platform owner can still report across companies. The cost is discipline: every query must carry the right filter — which is exactly why the filters are centralised in the access policy and proved by the audit.

What was the hardest bug?

One comparison that was wrong in both directions: the ticket detail check compared tenant_id, which on a ticket is the renter's organisation. It refused staff every ticket in their own buildings while allowing a rival's. The fix was to follow the building path.

Why NestJS and Prisma?

NestJS guards and decorators make access control declarative, route by route — the problem this project has. Prisma gives typed queries and migrations, and parameterises every query, so SQL injection is not reachable.

How do sessions work, and why sessionStorage?

A login creates a session row and two tokens: an access token for requests and a refresh token that is rotated on every refresh. sessionStorage is per tab: you can hold a manager and a renter in two tabs, and closing the tab ends the session.

Is the AI reliable?

It is not trusted. Any value outside the application's own lists discards the answer and a keyword classifier takes over; the technician comes from ticket history, never from the model; the space finder's model never selects data, so prompt injection cannot reach anything beyond the public listings. Without an API key the business still runs: tickets are classified by keywords and the space finder still searches.

Why no online payment?

It is a requirement taken from the market: rent here is paid by cheque or bank transfer, so the platform records payments and finance confirms them, with the name of who confirmed. A payment gateway would add cost and solve a problem this market does not have.

Why a receptionist role?

Because a lease is concluded in person: a confirmation call, then a site visit, then documents. The role exists to carry those steps, and each step is a booking state.

How is double booking prevented?

An overlap check runs before the booking is written, and the head count is checked against the room's capacity. A slot that overlaps a live booking is refused, not warned about afterwards.

Why Scrum?

Because the requirements moved: the specification asked for one level of multi-tenancy and the business needed two. Short sprints with a working increment reviewed by the company let that be discovered early.

How would it scale?

Sessions are database rows, not server memory, so a second API container can run behind nginx. Three things would then move to a shared store such as Redis: the rate-limit counters, the small search cache, and the Socket.IO rooms (an adapter, so a notification reaches a browser connected to the other container). With the demo volume pages load in under two seconds, helped by the client sharing identical requests in flight. For real volume, the next steps are server-side pagination on the large lists (bookings, invoices, which are loaded whole today) and indexes on the foreign keys the access filters follow — PostgreSQL does not create those automatically.

What is not finished?

The document upload interface (the server side works), scanned documents (they need a vision model), the external marketplace adapters (vendor APIs pending), the Google Business profile (needs the owner's verified account), production email (Brevo needs the server's address allow-listed; a fallback is active), and a mobile app (out of scope; the site is responsive). None of these blocks a business journey.

What would you do next?

The document intake interface with a vision model; extending the assistant to the renter's own data with a full prompt-injection test suite; replacing the triage heuristic with a model trained on ticket history, measured against the heuristic; and a mobile app for technicians.

PART 10

Glossary

The words used in this guide

Term	Meaning
Access token / refresh token	Two signed JWTs. The access token proves who you are on each request; the refresh token gets a new pair when it expires.
AccessPolicyService	The class in common/services that turns a user into database filters for spaces, floors, buildings and bookings.
Booking application	A request for a space, sent from the public map; accepted or refused by the manager.
CLIENT / RENTER	The two kinds of organisation: a property company (landlord) or a company renting space.
Controller / service / DTO	NestJS building blocks: routes, business rules, validated input.
Cross-tenant	Crossing from one organisation's data into another's — what isolation prevents.
Guard	A NestJS class that decides whether a request may reach a route (JwtAuthGuard, RolesGuard).
Interceptor	A NestJS class that wraps every response (ResponseInterceptor) or measures it (PerformanceInterceptor).
Intake	The in-person sequence before a lease: call, visit, documents, finalize.
JWT	JSON Web Token: a signed piece of text carrying the user id and session id.
Migration	A versioned SQL change to the database schema, applied in order by Prisma.
Multi-tenancy	Many customer organisations served by one installation, each seeing only its own data.
nginx	The web server in front: serves the React files and forwards API paths to NestJS.
Prisma	The database toolkit: schema, typed client, migrations.
QAR	Qatari riyal, the currency of every amount.
Seed	The script that fills the database with demo data (npm run db:seed:full).
Socket.IO	The WebSocket library that pushes notifications to the browser live.
Swagger	The interactive API documentation at /api.
Tenant	In the code: one customer organisation of the platform (landlord or renter). In everyday English: a renter.
TanStack Query	The React library that fetches, caches and refreshes server data.
Zustand	A small React state library; holds the signed-in user and the theme.